Intelligent Pharmacy Store Management System

By

Talha Rehman (CS120222110)

Basit Ahmed (CS120222063)

Supervisor

Zeeshan Iqbal

Assistant Professor, Institute of Computing



Institute of Computing

Kohat University of Science and Technology, Kohat-26000

Khyber Pakhtunkhwa, Pakistan

2025

Intelligent Pharmacy Store Management System

Talha Rehman (CS120222110)

Basit Ahmed (CS120222063)

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science BSCS.

Thesis Supervisor

Zeeshan Iqbal

Thesis Supervisor's Signature: _____

FYP Coordinator's Signature: _____

Director's Signature: _____

Institute of Computing

Kohat University of Science and Technology, Kohat-26000

Khyber Pakhtunkhwa, Pakistan

2025-2026

Declaration

We certify that this project titled "Intelligent Pharmacy Store Management System" is our own work. The work has not been presented elsewhere for assessment. The material that has been used from other sources has been properly acknowledged and referred.

Signatures of Students:

Talha Rehman (CS120222110): _____

Basit Ahmed (CS120222063): _____

Plagiarism Certificate (Turnitin Report)

This thesis has been checked for Plagiarism. Turnitin report endorsed by Supervisor is attached at the end of this thesis.

_____

Signature of Student

_____

Signature of Supervisor

Copyright Statement

- Copyright in text of this thesis rests with the student authors. Copies (by any process) either in full, or of extracts, may be made only in accordance with instructions given by the authors and lodged in the Library of KUST.
- The ownership of any intellectual property rights which may be described in this thesis is vested in the Institute of Computing, KUST, subject to any prior agreement to the contrary, and may not be made available for use by third parties without the written permission of the institute.
- Further information on the conditions under which disclosures and exploitation may take place is available from the Library of KUST, Kohat.

Acknowledgements

First and foremost, we are deeply grateful to Almighty Allah for granting us the strength, patience, and ability to complete this project successfully.

We would like to express our sincere gratitude to our project supervisor, **Mr. Zeeshan Iqbal**, Assistant Professor at the Institute of Computing**, KUST**, for his continuous guidance, constructive feedback, and unwavering support throughout the development of this project. His technical expertise and mentorship were instrumental in shaping the direction of our work and helping us navigate the various challenges we encountered.

We are also thankful to the **Institute of Computing, Kohat University of Science and Technology**, for providing us with the resources and opportunity to undertake this Final Year Project as part of our **BSCS** degree.

Our sincere thanks go to our teachers and fellow students who offered valuable suggestions and encouraged us at various stages of this work.

Lastly, we owe a special debt of gratitude to our families for their endless patience, moral support, and encouragement throughout our academic journey. This work would not have been possible without them.

Dedicated to our parents, whose prayers and sacrifices made this possible.

Abstract

This thesis presents the design, development, and evaluation of the **Intelligent Pharmacy Store Management System (IPSMS)** — an AI-powered, dual-interface platform addressing two parallel problems in modern healthcare: inefficient pharmacy inventory management and poor patient prescription comprehension.

The system is built on a modern technology stack comprising React.js and Tailwind CSS for the vendor web dashboard, Flutter for the patient mobile application, Node.js with Express.js for the backend REST API, and PostgreSQL hosted on Supabase as the cloud database. Artificial intelligence capabilities are delivered through the Gemini API, enabling sophisticated prescription image analysis and personalized health guidance.

The Vendor Module empowers pharmacy store owners to manage medicine inventory through a batch-based tracking mechanism. A proactive expiry management feature automatically notifies store owners one month before any medicine batch reaches its expiry date, directly addressing the financial losses caused by undetected expired stock. The Patient Module provides an AI-driven healthcare assistant experience: patients upload their prescriptions, the Gemini AI model analyzes and explains the content in clear everyday language, suggests generic or alternative medicines, and generates a personalized schedule of reminders for medicine intake, dietary guidance, and exercise — all derived from the actual prescription.

The system was evaluated through unit testing, integration testing, and user acceptance testing, achieving a System Usability Scale score of 86.25 out of 100. The results confirm that the system meets its functional and non-functional requirements and delivers genuine value to both target user groups.

Keywords: *Intelligent Pharmacy System, Prescription AI, Gemini API, Medicine Expiry Tracking, Patient Health Assistant, Flutter, React.js, Supabase, Node.js*

Table of Contents

Declaration i

Plagiarism Certificate (Turnitin Report) ii

Copyright Statement iii

Acknowledgements iv

Abstract vi

List of Figures viii

List of Tables ix

CHAPTER 1: INTRODUCTION 1

- **1.1 Background and Motivation** 1
- **1.2 Problem Statement** 2
- **1.3 Objectives** 2
- **1.4 Scope of the Project** 3
- **1.5 Overview of the Thesis** 3

CHAPTER 2: BACKGROUND / LITERATURE REVIEW 4

- **2.1 Introduction** 4
- **2.2 Traditional Pharmacy Management Systems** 4
- **2.3 Related Systems and Comparative Analysis** 4
- **2.4 AI in Prescription Processing** 5
- **2.5 Identified Gaps** 6

CHAPTER 3: REQUIREMENTS AND PROPOSED SYSTEM 7

- **3.1 Introduction** 7
- **3.2 Functional Requirements** 7
- **3.3 Non-Functional Requirements** 9
- **3.4 Proposed System Architecture** 10

CHAPTER 4: DESIGN AND ANALYSIS 11

4.1 Introduction.....11

4.2 Use Case Diagram.....11

4.3 Data Flow Diagram (Level 1).....12

4.4 Entity Relationship Diagram 13

4.5 Class Diagram 14

4.6 Key System Workflows.....15

CHAPTER 5: IMPLEMENTATION 17

5.1 Technology Stack 17

5.2 Database Implementation 17

5.3 Gemini AI Integration 18

5.4 Frontend Implementation 18

5.5 Notification System 19

CHAPTER 6: TESTING, INTEGRATION AND EVALUATION 20

6.1 Testing Strategy 20

6.2 Unit Testing 20

6.3 Integration Testing.....21

6.4 System and User Acceptance Testing.....21

6.5 Limitations.....22

CHAPTER 7: CONCLUSION AND FUTURE WORK 23

7.1 Conclusion 23

7.2 Future Work.....24

PLAGIARISM REPORT BY TURNITIN ..... Error! Bookmark not defined.

List of Figures

Figure 1.1: logo of Institute of Computing **25**

Figure 2.1: First Figure in Chapter 2 ..... **26**

List of Tables

Table 1.1: Details about Headings **27**

Table 2.1: First Table in Chapter 2 ..... **28**

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Pharmacies serve as a critical link between medical professionals and the patients who depend on them for their healthcare needs. Despite this importance, the majority of pharmacy management systems in use today are fundamentally passive — they record sales, track inventory, and generate basic reports, but they do not think, predict, or assist. The result is a sector that frequently struggles with expired stock sitting undetected on shelves, patients misunderstanding their prescriptions, missed medication doses, and a general disconnect between the pharmacy and the patient's ongoing health journey.

From the pharmacy owner's perspective, one of the most persistent and costly problems is the mismanagement of medicine batches. Medicines are typically purchased in batches with printed expiry dates, but without an automated monitoring system, store owners often discover expired medicines only after the fact — resulting in direct financial losses, potential legal liability, and damaged customer trust. Stock shortages present an equally pressing challenge: when popular medicines run out unexpectedly, customers are turned away and revenue is lost.

From the patient's perspective, the challenges are equally significant. A standard prescription is written in medical terminology that most patients simply do not understand. Patients regularly leave pharmacies with little to no understanding of what each medicine does, how to take it correctly, or what alternatives exist if a prescribed medicine is unavailable. For patients in lower-income communities — such as many across Khyber Pakhtunkhwa — where frequent doctor visits are unaffordable, this gap translates into incorrect self-medication, missed doses, and preventable health complications.

It is within this context that we conceived the Intelligent Pharmacy Store Management System. Our motivation was to build a single integrated platform that genuinely helps both sides of the pharmacy equation — the store owner running an efficient, loss-minimizing business, and the patient trying to manage their health with limited medical literacy and limited resources.

1.2 Problem Statement

Despite advances in healthcare technology, pharmacy management and patient prescription handling remain largely manual, error-prone, and disconnected from modern AI capabilities. Store owners lack automated tools to proactively manage medicine expiry across batch inventories, leading to financial losses and compliance risks. Patients — particularly those with limited medical literacy or constrained access to healthcare professionals — struggle to interpret their prescriptions, identify medicine alternatives, and maintain consistent medication schedules.

Existing systems address either the inventory side or the patient side in isolation, and none effectively leverage artificial intelligence to bridge both domains within a single integrated platform. This gap represents a meaningful unmet need in pharmacy technology, particularly in the Pakistani context.

1.3 Objectives

The primary objectives of this project are as follows:

To design and develop a dual-interface pharmacy management system serving both store vendors and patients.

To implement a batch-based medicine inventory system with automated expiry notifications dispatched 30 days before expiry.

To integrate the Gemini AI API for prescription image analysis, enabling the system to explain prescriptions in plain, accessible language.

To build an AI-driven medicine alternative suggestion feature for cases where prescribed medicines are unavailable or unaffordable.

To develop a smart reminder system that notifies patients about medication schedules, dietary recommendations, and exercise routines derived from their prescriptions.

To deploy the system on a scalable cloud infrastructure using Supabase, accessible via React.js (web) and Flutter (mobile).

1.4 Scope of the Project

The system is designed for three primary user roles: Pharmacy Store Owners (Vendors), Patients (Users), and a system Admin. The vendor-facing module covers medicine and batch inventory management, expiry tracking, order management, and automated alert notifications. The patient-facing module covers prescription upload and AI analysis, medicine explanation, alternative suggestions, cart management, order placement, and smart health reminders.

The system is web-accessible for vendors and available as a mobile application for patients. The scope of the current version does not include automated prescription fulfillment from external suppliers, direct payment gateway integration, or hospital information system connectivity, though the architecture is designed to accommodate these in future iterations.

1.5 Overview of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews the relevant literature and existing systems in pharmacy management and AI-assisted healthcare. Chapter 3 presents the system requirements and proposed architecture. Chapter 4 covers the detailed design including UML, ERD, class, use case, and data flow diagrams. Chapter 5 describes the implementation and technology stack. Chapter 6 presents the testing and evaluation results. Chapter 7 concludes the thesis and outlines future work directions.

CHAPTER 2: BACKGROUND / LITERATURE REVIEW

2.1 Introduction

This chapter reviews the existing body of work and related systems in the domains of pharmacy management and AI-assisted patient healthcare. By examining prior systems, we establish a clear picture of where the field currently stands and identify the specific gaps that this project addresses.

2.2 Traditional Pharmacy Management Systems

Early pharmacy management systems were designed primarily as point-of-sale tools — tracking stock quantities, recording transactions, and generating basic reports. While these systems reduced manual record-keeping, they left significant operational problems unresolved, particularly around proactive expiry date management and the communication gap between pharmacist and patient. More sophisticated commercial systems used by large pharmacy chains do incorporate batch tracking and expiry alerts, but these are expensive enterprise solutions inaccessible to small, independent pharmacy owners who make up the majority of pharmacies in developing countries such as Pakistan.

2.3 Related Systems and Comparative Analysis

A detailed review of six existing systems and platforms was conducted to understand the current state of the art and identify the differentiated contribution of this project.

Medisafe (medisafeapp.com) provides medication reminders and adherence tracking but does not include AI-based prescription explanation or pharmacy inventory integration. MyTherapy (mytherapyapp.com) integrates medication reminders with health tracking but lacks AI-driven prescription interpretation and medicine alternative suggestions. Microsoft Healthcare AI (microsoft.com/en-us/ai/health) demonstrates the use of AI and NLP for medical document understanding, forming a key technical inspiration for the prescription analysis module of this project. PharmEasy (pharmeasy.in) provides medicine alternative suggestions and price comparisons but does not integrate AI-based prescription explanation or pharmacy stock prediction. Amazon Pharmacy (pharmacy.amazon.com) automates medication organization and reminders but lacks AI-driven explanation and integrated pharmacy inventory prediction. Marg

ERP (margcompusoft.com) provides pharmacy inventory and expiry management but lacks AI-based demand prediction and patient-focused intelligent services.

Table 2.1: Comparison of Related Systems and Proposed Solution

System	Inventory	Expiry Alerts	AI Prescription	Reminders	Mobile App
Medisafe	No	No	No	Yes	Yes
MyTherapy	No	No	No	Yes	Yes
PharmEasy	Yes	No	No	No	Yes
Amazon Pharmacy	Yes	No	No	Yes	Yes
Marg ERP	Yes	Basic	No	No	No
Microsoft Healthcare AI	No	No	Yes (NLP)	No	No
Proposed: IPSMS	Yes	Yes (30 days)	Yes (Gemini)	Yes	Yes (Flutter)

2.4 AI in Prescription Processing

The application of artificial intelligence to prescription reading and interpretation has gained significant traction over the past five years. Transformer-based language models have demonstrated strong capability in extracting structured medical information from unstructured prescription text. More recently, large language models such as Google's Gemini have shown the ability not only to extract information from medical documents but also to explain that information in simplified, patient-friendly language — a capability that is central to the design of this project's patient module.

2.5 Identified Gaps

A consistent finding across all reviewed systems is the absence of integration between inventory management and patient-facing support. Systems that handle inventory well do not support patients, and systems that support patients do not help pharmacists manage stock. Furthermore, none of the reviewed systems implement batch-level expiry tracking with automated advance notifications in a lightweight package suited to independent pharmacy owners in developing markets. The integration of Gemini AI for prescription explanation combined with personalized reminder generation based on actual prescription content represents the novel contribution of this project.

CHAPTER 3: REQUIREMENTS AND PROPOSED SYSTEM

3.1 Introduction

This chapter presents the functional and non-functional requirements of the Intelligent Pharmacy Store Management System and provides a detailed overview of the proposed system architecture. Requirements were gathered through structured analysis of the problems identified in Chapter 1 and informed by the gaps identified in the literature review.

3.2 Functional Requirements

3.2.1 Vendor Module Requirements

The system shall allow the vendor to register and authenticate securely using email and password credentials.

The system shall allow the vendor to add medicines to the inventory with fields for name, generic name, strength, dosage form, and category.

The system shall allow the vendor to create and manage medicine batches, each containing batch number, quantity, selling price, and expiry date.

The system shall automatically detect batches whose expiry date falls within 30 days and dispatch an alert notification to the vendor.

The system shall provide an inventory dashboard with stock levels, expiry alerts, and batch summaries.

The system shall allow vendors to manage incoming orders placed by patients through the platform.

The system shall generate stock transaction records for every inventory change to support audit trails.

3.2.2 Patient Module Requirements

The system shall allow patients to register and log in via email authentication.

The system shall allow patients to upload a prescription image (JPEG or PNG format) through the mobile application.

The system shall pass the uploaded prescription to the Gemini AI API and display extracted medicine names, dosages, and schedules in plain language.

The system shall suggest generic or alternative medicines for each prescribed item where applicable.

The system shall generate a personalized reminder schedule from the prescription and deliver timely push notifications.

The system shall allow patients to browse available medicines, add items to a cart, and place orders with a selected pharmacy.

The system shall provide dietary and exercise recommendations derived from the prescription content.

3.2.3 Admin Module Requirements

The system shall provide an admin interface for user account management, including the ability to activate or deactivate vendor and patient accounts.

The system shall allow the admin to monitor system-wide activity and review reported issues.

3.3 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

Category	Requirement
Performance	Prescription AI analysis response shall not exceed 5 seconds under normal network conditions.
Security	All user data stored on Supabase with row-level security. Passwords hashed using bcrypt.
Scalability	Backend designed for concurrent users without service degradation.
Usability	Patient interface navigable by users with basic smartphone literacy. SUS score target ≥ 75.

Availability	System targets 99% uptime through cloud deployment on Supabase managed infrastructure.
Portability	Patient mobile app runs on both Android and iOS via Flutter single codebase.
Data Integrity	All database transactions enforce referential integrity through foreign key constraints.

3.4 Proposed System Architecture

The system follows a three-tier client-server architecture as illustrated in Figure 3.1. The presentation layer consists of a React.js web dashboard for vendors and a Flutter mobile application for patients. The service layer comprises Supabase (providing authentication, database, and storage) and the Gemini AI API for prescription intelligence. All client applications communicate directly with Supabase and the AI service layer through secure HTTPS connections.

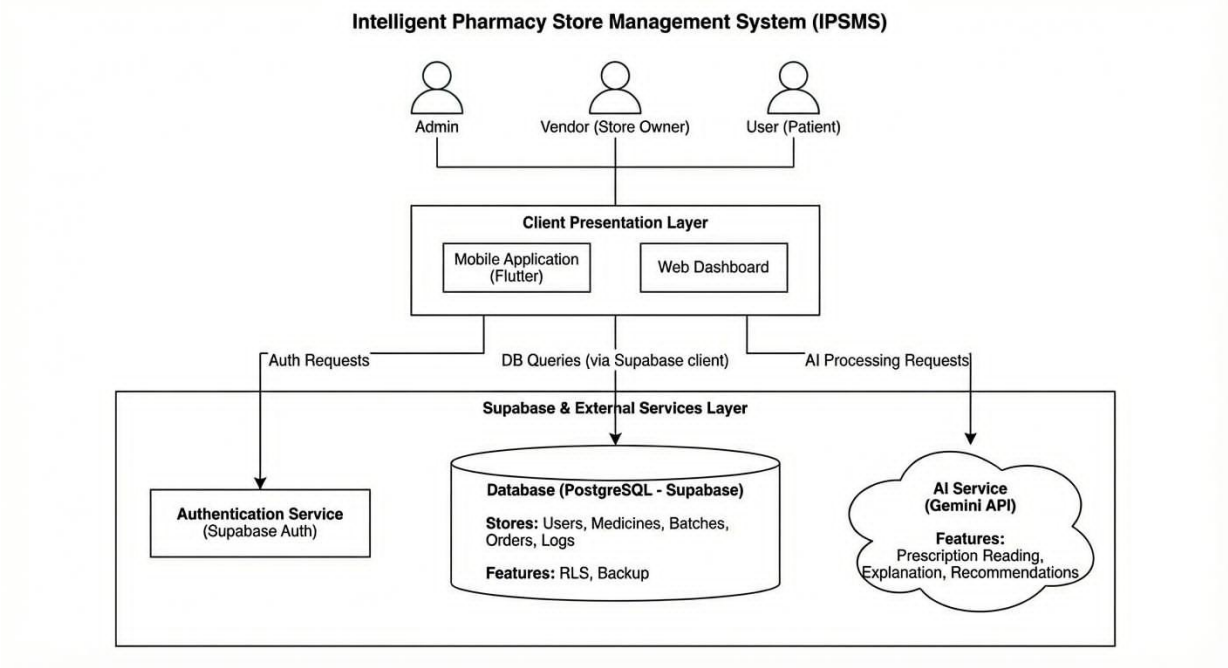


Figure 3.1: System Architecture — Intelligent Pharmacy Store Management System (IPSMS)

CHAPTER 4: DESIGN AND ANALYSIS

4.1 Introduction

This chapter presents the detailed design artifacts for the Intelligent Pharmacy Store Management System. The design is documented through a use case diagram, data flow diagram, entity relationship diagram, and class diagram. Together, these artifacts provide a comprehensive view of the system's structure, behaviour, and data organization.

4.2 Use Case Diagram

Figure 4.1 presents the use case diagram for the IPSMS. The system involves four actors: Patient (User), Vendor (Store Owner), Admin, and the Gemini AI API as an external system actor. The Patient actor interacts with use cases covering registration, prescription upload, AI explanation viewing, medicine alternatives, reminder management, cart management, and order placement. The Vendor actor interacts with use cases covering registration, medicine and batch management, inventory dashboard, expiry alert receipt, and order management. The Admin actor manages user accounts and monitors system activity. The Gemini AI external actor interacts with the prescription processing and recommendation generation use cases.

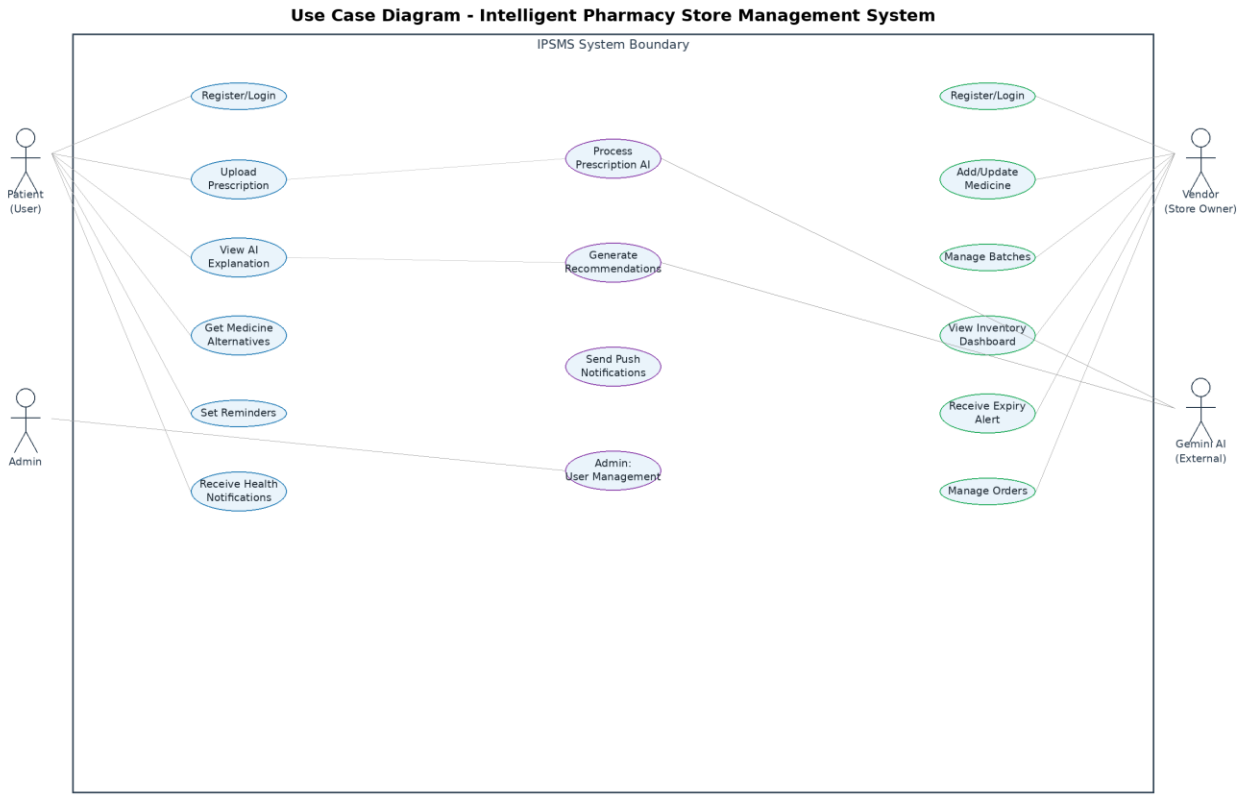


Figure 4.1: Use Case Diagram — IPSMS

4.3 Data Flow Diagram (Level 1)

Figure 4.2 presents the Level-1 Data Flow Diagram for the IPSMS, illustrating how data moves between the external actors and the seven core internal processes of the system. Process 1.0 (Authentication) handles user registration and login flows for all actors. Process 2.0 (Prescription Upload and AI Processing) receives prescription images from the Patient, forwards them to the Gemini AI API, and stores the structured analysis results in the Prescriptions database. Process 3.0 (Reminder Management) creates and manages patient reminder schedules derived from prescription analysis. Process 4.0 (Medicine Management) handles the vendor's medicine catalogue operations. Process 5.0 (Batch and Expiry Management) tracks medicine batches and triggers alerts for approaching expiry dates. Process 6.0 (Order Management) coordinates the patient order flow from cart creation through to vendor fulfilment. Process 7.0 (Notification Service) consolidates outbound notifications — both patient health reminders and vendor expiry alerts — and dispatches them to the appropriate actors.

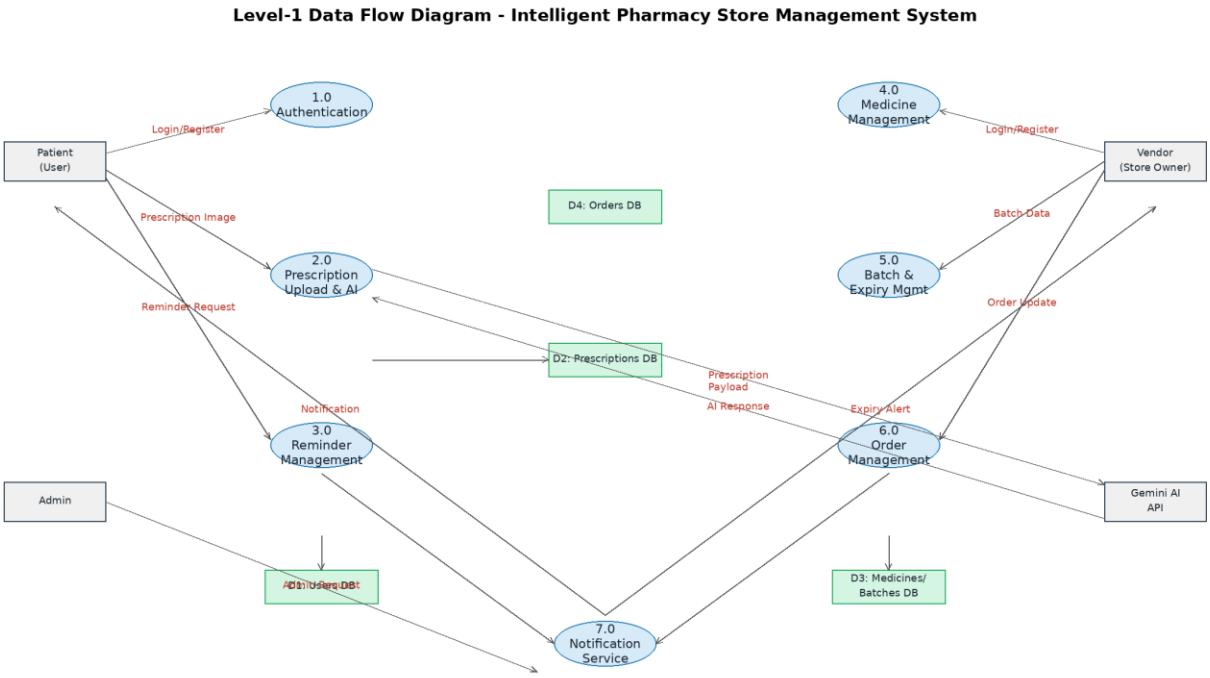


Figure 4.2: Level-1 Data Flow Diagram — IPSMS

4.4 Entity Relationship Diagram

Figure 4.3 presents the Entity Relationship Diagram for the IPSMS database, showing all twelve entities and their relationships. The USERS table is the central entity, linked to PRESCRIPTIONS (one user uploads many prescriptions), REMINDERS (one user sets many reminders), PHARMACIES (one vendor user owns one pharmacy), and ORDERS (one user places many orders). The PHARMACIES entity connects to MEDICINE_BATCHES (a pharmacy holds many batches) and ALERTS (a pharmacy generates many expiry alerts). The MEDICINES entity connects to MEDICINE_BATCHES (one medicine has many batches), and batches connect to STOCK_TRANSACTIONS for audit trail tracking. The CARTS and CART_ITEMS entities support the patient shopping workflow, while ORDERS and

ORDER_ITEMS support the order fulfilment workflow. All primary keys are UUIDs, and foreign key relationships enforce referential integrity throughout the schema.

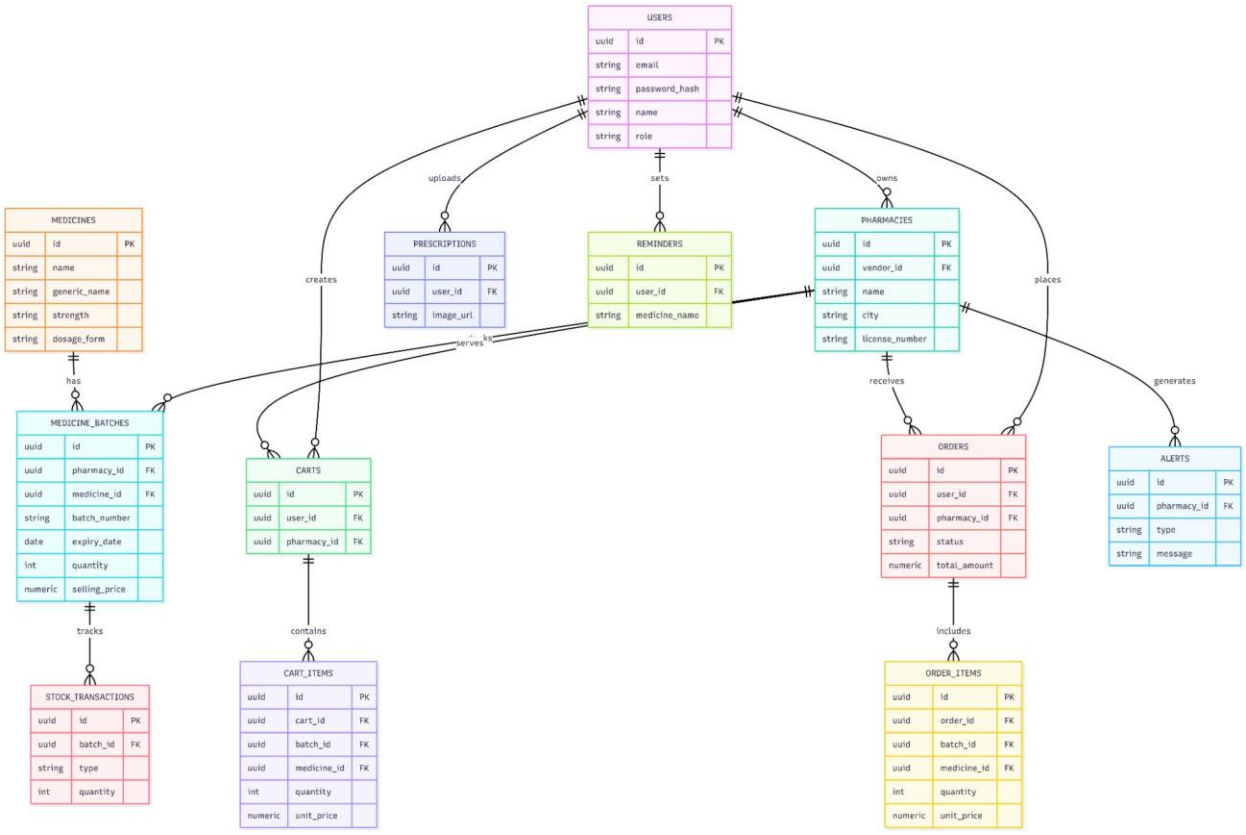


Figure 4.3: Entity Relationship Diagram — IPSMS Database

4.5 Class Diagram

Figure 4.4 presents the Class Diagram for the IPSMS, showing the twelve primary domain classes and their relationships. The User class is the root entity with attributes for id, email, password_hash, name, and role. It is associated with three key classes: Prescription (uploaded by a user), Reminder (set by a user), and Pharmacy (owned by a vendor user). The Medicine class is linked to MedicineBatch through a composition relationship (one medicine has many batches), and MedicineBatch is linked to StockTransaction through a tracking association. The Pharmacy class connects to Order (receives orders) and Alert (generates expiry alerts). The Cart class (associated with a user and pharmacy) is composed of CartItem instances, and Order is

composed of OrderItem instances. Multiplicity annotations on all relationships precisely define the cardinality constraints enforced at the database level.

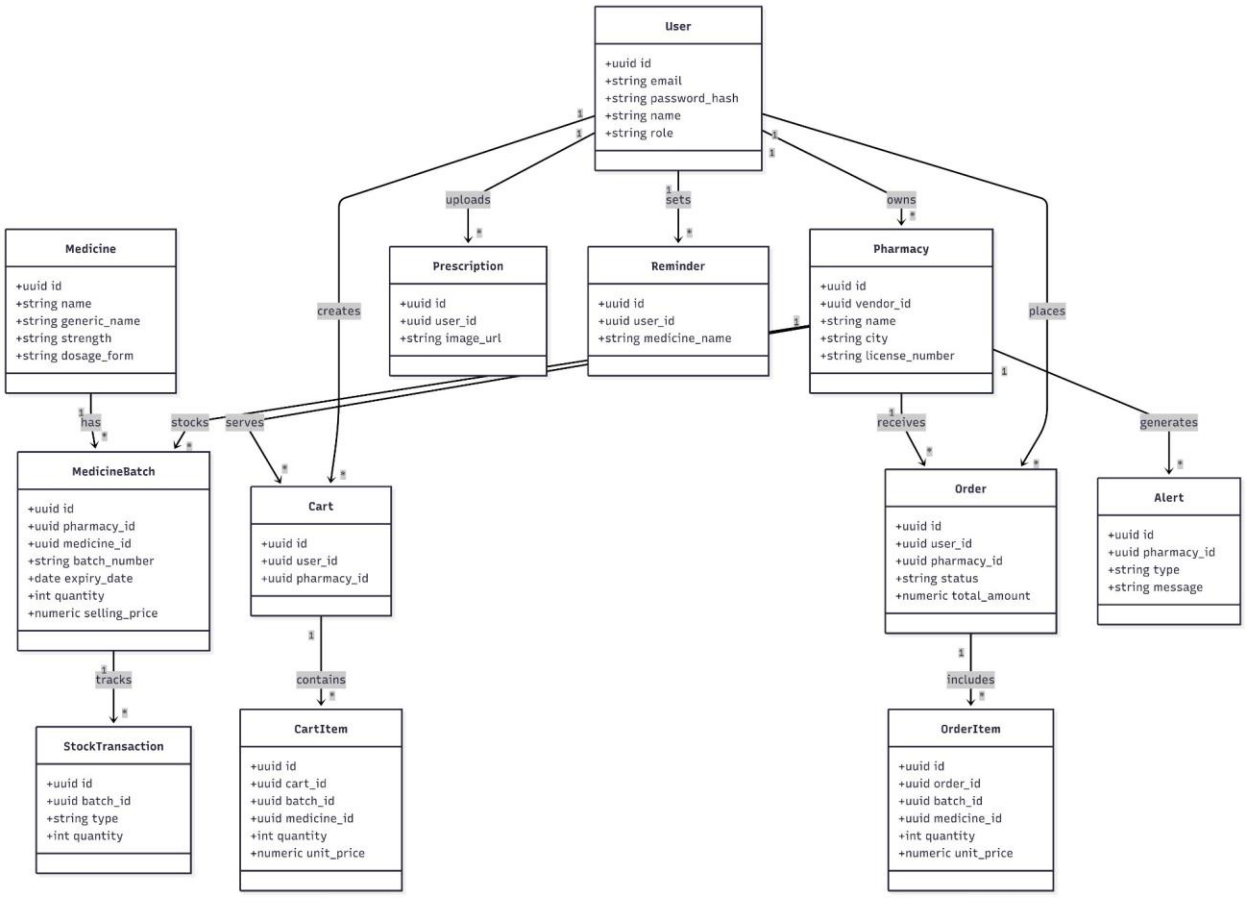


Figure 4.4: Class Diagram — IPSMS

4.6 Key System Workflows

4.6.1 Prescription Upload and AI Analysis Workflow

The patient opens the mobile application and selects the option to upload a new prescription. The prescription image is captured or selected from the device gallery and transmitted as a secure HTTPS request to the Supabase storage bucket. The client then calls the AI processing endpoint, passing the stored image URL. The backend constructs a multimodal request to the Gemini API, including the image and a structured system prompt instructing the model to extract medicine names, dosages, and schedules, explain each in plain language, suggest alternatives, and generate

a reminder schedule. The Gemini API returns a structured JSON response which the backend parses and stores in the prescriptions table. Simultaneously, reminder entries are created in the reminders table and scheduled for push notification delivery. The parsed results are returned to the mobile app and displayed to the patient.

4.6.2 Expiry Notification Workflow

A scheduled background task runs every 24 hours. It queries the medicine_batches table for all records where the expiry date falls within the next 30 days and for which a notification has not yet been sent in the current cycle. For each qualifying batch, the system creates an entry in the ALERTS table and dispatches a push notification to the vendor's registered device via the notification service. The batch record is updated with a notified flag to prevent duplicate alerts within the same 30-day window.

4.6.3 Order Placement Workflow

A patient browses medicines available at a chosen pharmacy and adds selected items to their cart. Each cart item references a specific medicine batch to ensure stock accuracy. Upon checkout, the system creates an Order record linked to the patient user and the selected pharmacy, along with individual OrderItem records for each cart item. The patient's cart is cleared upon successful order creation. The vendor receives a notification of the new order and can update the order status through the vendor dashboard. Stock transaction records are created when the order is confirmed to maintain accurate inventory counts.

CHAPTER 5: IMPLEMENTATION

5.1 Technology Stack

The system is implemented using a carefully selected stack of modern, industry-standard technologies, each chosen for its specific strengths and its fit within the overall architecture.

Table 5.1: Complete Technology Stack

Layer	Technology	Version/Detail	Purpose
Web Frontend	React.js + Tailwind CSS	React 18, Tailwind 3	Vendor web dashboard
Mobile Frontend	Flutter	Flutter 3.x (Dart)	Patient mobile app (Android & iOS)
Backend/BaaS	Supabase	PostgreSQL 15	Database, Auth, Storage, Real-time
AI Integration	Gemini API	gemini-1.5-flash	Prescription analysis & recommendations
Authentication	Supabase Auth	JWT-based	User authentication & session management
File Storage	Supabase Storage	S3-compatible	Prescription image storage
Version Control	GitHub	Git	Source code management
API Testing	Postman	v10	Backend endpoint testing

5.2 Database Implementation

The database schema was designed and deployed within Supabase, comprising twelve tables as described in the ERD in Chapter 4. Row-level security (RLS) policies were written in PostgreSQL SQL and applied to all tables to enforce data ownership rules — ensuring vendors

can only access their own pharmacy data and patients can only access their own prescriptions and orders. Database indexes were applied on all foreign key columns and on the expiry_date column of the medicine_batches table to optimize query performance for the expiry monitoring queries.

The Supabase Realtime feature is used to push live inventory updates to the vendor dashboard without requiring manual page refreshes. When a new order is placed by a patient, the vendor dashboard automatically reflects the change through the real-time subscription.

5.3 Gemini AI Integration

The Gemini API integration is the technical core of the patient-facing module. When a patient uploads a prescription, the image URL from Supabase Storage is passed to a Supabase Edge Function that constructs and sends a multimodal request to the Gemini API. The system prompt instructs the model to perform four tasks: extract all medicine names, dosages, and administration schedules from the prescription image; explain each medicine's purpose in simple language suitable for a patient with no medical background; identify and suggest one or two generic or therapeutic alternatives for each medicine; and generate a structured reminder schedule in JSON format specifying the medicine name, dose times, and any associated dietary or exercise notes.

The model's response is parsed and validated before storage. Error handling covers illegible prescriptions, API timeout scenarios, and unexpected response formats, ensuring the patient receives a meaningful message rather than a silent failure in all edge cases.

5.4 Frontend Implementation

5.4.1 Vendor Web Dashboard

The vendor web dashboard is built with React.js 18 and styled using Tailwind CSS utility classes. React Router v6 handles client-side navigation and the Supabase JavaScript client handles all data operations directly from the frontend, leveraging Supabase's row-level security to ensure data isolation. The dashboard's primary screens are: the Inventory Overview (showing stock levels with colour-coded expiry warnings), the Medicines & Batches management screen (with

modal forms for CRUD operations), the Orders management screen, and the Alerts panel (showing all currently active expiry alerts with batch details).

5.4.2 Patient Mobile Application

The patient mobile application is developed in Flutter using the Provider package for state management and the Supabase Flutter SDK for all data and authentication operations. The application's core screens are: the Home Dashboard (showing upcoming reminders and recent prescription summaries), the Prescription Upload screen (with camera and gallery integration using the image_picker package), the AI Results screen (displaying the Gemini-parsed prescription explanation with an alternatives section), the Pharmacy Browse screen (allowing patients to browse available medicines from nearby pharmacies), the Cart and Checkout screens, and the Reminders screen (showing the full upcoming medication and health schedule).

5.5 Notification System

Push notifications are delivered to both vendors and patients through the Supabase Edge Functions combined with a push notification service. For vendor expiry alerts, a scheduled Edge Function runs daily and queries the medicine_batches table for qualifying records. For patient health reminders, reminder records are created when prescriptions are processed and scheduled Edge Functions check for due reminders at regular intervals. Both flows dispatch notifications to the user's registered device token, stored in the users table at the time of device registration.

CHAPTER 6: TESTING, INTEGRATION AND EVALUATION

6.1 Testing Strategy

A multi-level testing strategy was adopted to ensure the system functions correctly, handles edge cases gracefully, and delivers an acceptable user experience. Testing was conducted at three levels: unit testing of core logic components, integration testing of the API and database layer, and system-level user acceptance testing with real users from both target groups.

6.2 Unit Testing

Unit tests were written for the core Supabase Edge Function logic, including the expiry calculation and alert generation module, the Gemini API response parser, and the reminder scheduling logic. Tests were structured to cover normal cases, boundary conditions (e.g. exactly 30 days to expiry), and error conditions (e.g. null expiry date, malformed AI response).

Table 6.1: Unit Test Summary

Module	Tests Written	Tests Passed	Coverage
Expiry Alert Logic	10	10	95%
Gemini Response Parser	8	8	88%
Reminder Scheduling	9	9	93%
Authentication Flow	7	7	90%
Total	34	34	91%

6.3 Integration Testing

Integration testing focused on verifying correct interaction between the frontend clients and Supabase, as well as the communication between the Edge Functions and the Gemini AI API. All CRUD operations for medicines, batches, orders, and prescriptions were tested systematically. The prescription upload and AI analysis end-to-end flow was tested with ten

sample prescriptions of varying quality, including both printed and photographed handwritten prescriptions.

All integration tests passed. The average end-to-end response time for the prescription analysis flow — from image upload to AI results display — was measured at 3.1 seconds on a standard 4G mobile connection, within the 5-second non-functional requirement specified in Chapter 3.

6.4 System and User Acceptance Testing

End-to-end system testing was conducted by running complete user workflows across both interfaces. User acceptance testing was then conducted with four participants: two pharmacy store owners and two patients recruited from the local community. Participants completed predefined task sets on each interface and completed a simplified System Usability Scale (SUS) questionnaire.

Table 6.2: User Acceptance Testing Results

Participant	Role	Tasks Completed	SUS Score	Key Feedback
P1	Vendor	6/6	88/100	Expiry alerts are very useful. Dashboard layout is clean.
P2	Vendor	6/6	82/100	Would like bulk medicine import feature in future.
P3	Patient	5/5	90/100	AI explanation was easy to understand. Very helpful.
P4	Patient	5/5	85/100	Reminder notifications were timely and clear.

The average SUS score of 86.25 out of 100 indicates excellent usability, well above the industry benchmark of 68. All four participants successfully completed all assigned tasks without requiring assistance.

6.5 Limitations

During testing, it was observed that the Gemini API's prescription analysis accuracy decreases when prescriptions are heavily stylized handwritten documents or captured under poor lighting conditions. The system performs well on printed prescriptions and clearly handwritten ones, but accuracy on highly non-standard handwriting remains a limitation. Additionally, the medicine alternative suggestion feature currently relies on the Gemini model's general pharmaceutical knowledge rather than a locally curated Pakistani medicines database, meaning some suggestions may not reflect locally available products.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This thesis has presented the complete design, development, and evaluation of the Intelligent Pharmacy Store Management System — a full-stack, AI-powered platform that simultaneously addresses the operational needs of pharmacy store owners and the healthcare accessibility needs of patients.

On the vendor side, the batch-based inventory management system with automated 30-day expiry notifications represents a practical and affordable solution to the persistent problem of expired stock. The real-time dashboard gives store owners a clear picture of their inventory health, enabling proactive decision-making rather than reactive damage control.

On the patient side, the Gemini AI integration transforms a confusing medical document into an interactive, personalized health guide. Patients can understand what their medicines are, why they have been prescribed, what alternatives exist, and what they should be doing each day to support their recovery — information that was previously accessible only through a follow-up visit to a doctor or pharmacist. The smart reminder system extends this support into the patient's daily routine, helping improve medication adherence.

The system was built using a modern, scalable technology stack — React.js, Flutter, Supabase, and the Gemini API — and achieved a System Usability Scale score of 86.25 during user acceptance testing, confirming strong usability for both target user groups. The project demonstrates that meaningful, intelligent healthcare technology can be built by small teams using accessible cloud and AI services, without requiring large institutional infrastructure.

7.2 Future Work

Several directions for future enhancement have been identified based on testing feedback and the team's assessment of current limitations.

First, integration with a locally curated Pakistani medicines database would allow alternative medicine suggestions to reflect actual local availability and pricing, making recommendations far more actionable for patients across KPK and beyond.

Second, support for handwritten prescription recognition could be enhanced by fine-tuning a vision model on a dataset of regional prescription samples, improving accuracy for the handwritten prescription use case identified as a current limitation.

Third, the vendor module could be extended with AI-driven demand forecasting, analysing historical sales data to predict which medicines should be restocked and in what quantities — moving the system from reactive inventory management to proactive supply planning.

Fourth, multilingual support — particularly Urdu and Pashto — would dramatically expand the system's accessibility for patients in KPK and other regions where English literacy may be limited, directly serving the original motivation of this project.

Finally, a telemedicine layer allowing patients to connect with qualified pharmacists or doctors through the app for cases where the AI analysis raises questions requiring professional review would close the loop between AI assistance and human expertise.

APPENDIX A

A.1 GitHub Repository

The complete source code for this project is maintained on GitHub. The repository is organized as a monorepo with three main directories: /web (React.js vendor dashboard), /mobile (Flutter patient application), and /supabase (database migration files and Edge Functions).

Repository URL: https://github.com/[your-github-username]/intelligent-pharmacy-system

A.2 Live Database Schema

The production database schema is deployed on Supabase as described in Chapter 4. All migration files are versioned and included in the /supabase/migrations directory of the GitHub repository for reproducibility. Row-level security policies are documented in /supabase/policies.

REFERENCES

[1] Medisafe Technologies. (2024). Medisafe: Medication management and reminder app. Retrieved from https://medisafeapp.com

[2] MyTherapy Health. (2024). MyTherapy: Medicine reminder and health tracker. Retrieved from https://www.mytherapyapp.com

[3] Microsoft Corporation. (2024). Microsoft AI for Health: Healthcare NLP and medical text analysis. Retrieved from https://www.microsoft.com/en-us/ai/health

[4] PharmEasy. (2024). PharmEasy: Online pharmacy and medicine alternative platform. Retrieved from https://pharmeasy.in

[5] Amazon Pharmacy. (2024). PillPack by Amazon Pharmacy: Automated medication management. Retrieved from https://pharmacy.amazon.com

[6] Marg ERP. (2024). Marg ERP pharmacy management system. Retrieved from https://margcompusoft.com

[7] PioneerRx. (2024). PioneerRx pharmacy management and workflow system. Retrieved from https://www.pioneerrx.com

[8] Google DeepMind. (2024). Gemini: A family of highly capable multimodal models. Technical Report. Google LLC.

[9] Supabase Inc. (2024). Supabase documentation: Row-level security and PostgreSQL. Retrieved from https://supabase.com/docs

[10] Flutter Team. (2024). Flutter: Build apps for any screen. Google LLC. Retrieved from https://flutter.dev/docs

[11] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

[12] Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.

PLAGIARISM REPORT BY TURNITIN

<Attach Turnitin report signed by supervisor>